
LEMON: Learning Executable Multi-Agent Orchestration via Counterfactual Reinforcement Learning

Xudong Chen Yixin Liu Hua Wei Kaize Ding

Abstract

Large language models (LLMs) have become a strong foundation for multi-agent systems, but their effectiveness depends heavily on orchestration design. Across different tasks, role design, capacity assignment, and dependency construction jointly affect both solution quality and execution efficiency. Existing approaches automate parts of this design process, yet they often optimize these decisions partially or sequentially, and rely on execution-level feedback that provides limited credit assignment for local orchestration decisions. We propose LEMON (Learning Executable Multi-agent Orchestration via Counterfactual Reinforcement Learning), an LLM-based orchestrator that generates an executable orchestration specification. The specification integrates task-specific roles, customized duties, capacity levels, and dependency structure into a single deployable system. To train the orchestrator, we augment the orchestration-level GRPO objective with a localized counterfactual signal that edits role, capacity, or dependency fields and applies the resulting reward contrast only to the edited spans. Experiments on six reasoning and coding benchmarks, including MMLU, GSM8K, AQuA, MultiArith, SVAMP, and HumanEval, show that LEMON achieves state-of-the-art performance among the evaluated multi-agent orchestration methods. Our code is available at <https://anonymous.4open.science/r/LEMON-B23C>.

1 Introduction

Recent advances in large language models (LLMs) have driven growing interest in LLM-based multi-agent systems [1, 2, 3]. By coordinating multiple agents, these systems can leverage tool use, decomposition, verification, and collaborative reasoning to tackle tasks that are difficult for a single agent alone. However, their effectiveness depends not only on the capacity of individual agents, but also on the orchestration scheme that determines what task-specific roles agents play, what capacity levels they are assigned, and how information flows among them [4]. A fixed orchestration scheme that works well for one task may be redundant for a simple query, underpowered for a harder one, or poorly coordinated when the required information flow changes [5, 6]. Manually designing a separate scheme for each task is therefore impractical, motivating automatic methods for constructing task-adaptive orchestration schemes for multi-agent systems.

Recent work has begun to move MAS orchestration beyond fixed manual designs by adapting different parts of the orchestration. Topology-learning methods [7, 8, 9] optimize information-flow structures, but usually over a specified set of agents or role profiles. Workflow-orchestration methods learn executable procedures for decomposing tasks, invoking workers, and routing intermediate context [10, 11, 12], while largely inheriting worker definitions and capacity assignment from the available workflow space. Routing and team-selection methods further adapt role assignment, architecture search, or model routing [13, 14], but often connect these decisions through staged or cascaded procedures rather than one unified executable specification. This separation is limiting because the role design, capacity assignment, and dependency construction of a multi-agent system

are coupled by execution and together determine whether the resulting orchestration is both effective and efficient. Overall, these methods make MAS orchestration more adaptive, but leave the following two challenges open.

Challenge 1: Decomposed design of executable orchestration specifications.

Existing methods often optimize only part of the orchestration design space at the level of individual design components, rather than at the level of the final executable specification, as illustrated by the decomposed generation path in Figure 1. As a result, task-specific role design, capacity assignment, and dependency construction are not usually treated as mutually dependent decisions of a single executable orchestration specification. This separation is limiting because the role, capacity, and dependency choices of a multi-agent system are coupled by execution and together determine whether the resulting orchestration is both feasible and efficient.

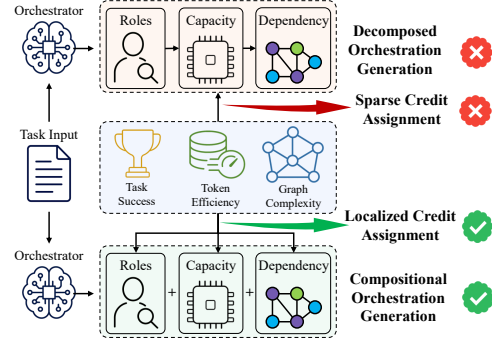


Figure 1: Comparison of decomposed and compositional orchestration generation under sparse and localized credit assignment.

Challenge 2: Sparse credit assignment for orchestration learning.

Learning executable orchestration specifications also raises a credit-assignment challenge. Most training signals for LLM-based orchestrators are attached to end-to-end execution outcomes, such as final task success, workflow utility, or overall performance–cost trade-offs. Such feedback can evaluate whether a generated orchestration succeeds as a whole, but it is too sparse to indicate which role, capacity, or dependency decisions are actually responsible for the outcome, as summarized by the sparse credit-assignment branch in Figure 1. As a result, a high-reward specification may contain both necessary and incidental choices, while a low-reward one may fail for only a small subset of local decisions. Directly reinforcing the whole specification therefore conflates necessary and incidental choices. It may reward or penalize local decisions simply because they appear in successful or failed executions, rather than because they are necessary for the final execution result.

To address these issues, we propose LEMON, an LLM-based framework for learning executable multi-agent orchestration. LEMON formulates MAS orchestration as **compositional executable orchestration generation**: for each input task, the orchestrator outputs a structured specification that determines the agents to execute, their task-specific roles, their capacity levels, and their information dependencies as one executable orchestration. This shifts optimization from isolated topology, workflow, or routing decisions to the executable specification that is actually deployed. To train this generator, LEMON introduces **localized counterfactual reinforcement learning**. In addition to an orchestration-level GRPO reward for task success and execution efficiency, it constructs local counterfactual edits to role, capacity, or dependency decisions and compares the edited specification with the original one. The resulting signal is applied only to the edited span of the generated specification, assigning credit to the changed orchestration decision rather than to the whole specification. Our contributions are summarized as follows:

- **Compositional executable orchestration generation.** We formulate LLM-based MAS orchestration as generating an executable orchestration specification that composes task-specific role designs, agent-level capacity profiles, and dependency structure.
- **Localized counterfactual reinforcement learning.** We address the sparse credit-assignment problem in executable orchestration learning by augmenting orchestration-level GRPO with a localized counterfactual signal. This signal converts execution-level reward differences into span-level supervision for the edited orchestration decision.
- **Empirical validation.** Experiments across reasoning and coding benchmarks show that LEMON improves over single-agent, fixed-structure MAS, topology-design, and adaptive orchestration baselines, with ablations validating both compositional executable orchestration generation and localized counterfactual supervision.

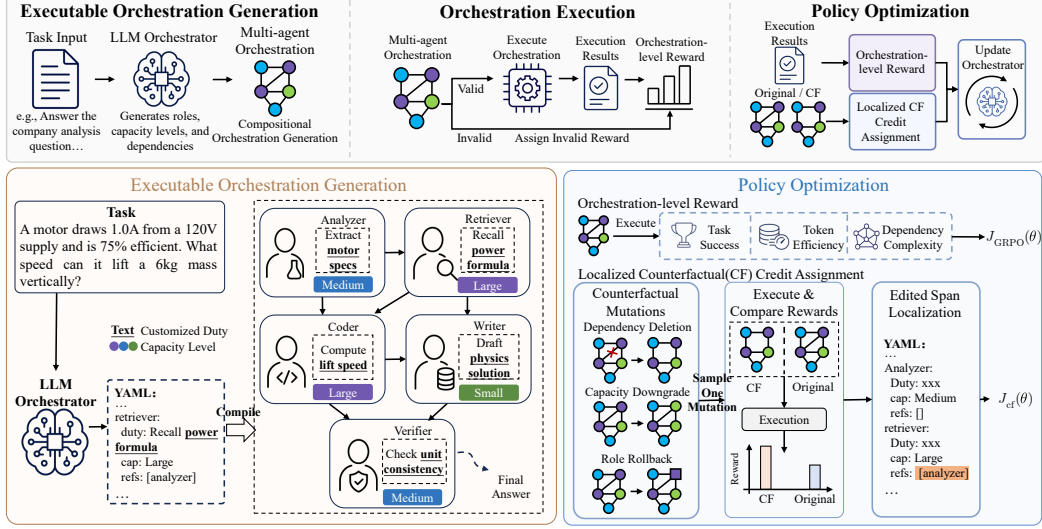


Figure 2: Overview of LEMON. The orchestrator generates an executable orchestration specification that composes task-specific roles, capacity levels, and dependency references, and is trained with orchestration-level GRPO and localized counterfactual credit assignment.

2 Related Work

Multi-Agent Collaboration and Orchestration Design. Recent work has moved LLM-based multi-agent systems beyond fixed manual designs by automating specific components of orchestration. Topology-oriented methods learn or search communication structures, e.g., sparse communication, graph pruning, or task-adaptive graph generation, but usually operate over predefined agents or role profiles [7, 9, 8, 15, 16]. Workflow-oriented methods search or generate executable agentic procedures, e.g., code-represented workflow search, natural-language orchestration, or difficulty-aware workflow construction, but typically rely on predefined worker interfaces, operators, or workflow primitives [17, 10, 11, 18]. Routing and team-construction methods adapt collaboration modes, agent sets, or heterogeneous LLM backbones, but often determine these choices through staged or cascaded controllers [13, 19, 14, 20]. Overall, these methods improve the adaptivity of multi-agent orchestration, yet they do not compositionally generate role specifications, capacity levels, and dependency structure as a single executable orchestration specification. In contrast, we formulate orchestration as compositional executable specification generation, where role specifications, capacity levels, and dependency structure are generated together in the same specification that is directly validated, compiled, and executed.

Reinforcement Learning for Multi-Agent Orchestration. Reinforcement learning has become an important mechanism for optimizing LLM-based multi-agent systems from execution feedback. Existing methods train policies with rewards defined over final task success, execution efficiency, preference alignment, model or tool usage, and topology complexity [11, 21, 22, 23, 24]. Recent topology-focused work further uses group-relative optimization to reduce reward noise and assign finer-grained credit to communication edges [25]. However, these signals are still largely assigned to complete workflows, trajectories, collaboration graphs, or orchestration specifications, leaving the necessity of specific dependency references, role specializations, and capacity levels underdetermined. LEMON differs by converting execution-level reward differences into localized counterfactual credit over edited orchestration fields.

3 Method

We present LEMON, an LLM-based orchestration policy for generating executable multi-agent orchestrations. Given an input task, LEMON generates an executable orchestration specification that jointly determines the agents to instantiate, their task-specific role specifications, their capacity

levels, and their dependency structure. The specification is serialized as a structured YAML object, validated against an execution schema, compiled into an orchestration graph, and then executed by worker agents. The orchestration policy is optimized with global orchestration-level GRPO over complete executed specifications and a localized counterfactual objective over edited orchestration fields. Together, this design composes role, capacity, and dependency decisions into one executable generation space, while assigning credit at both the complete-specification level and the edited-field level. Figure 2 illustrates the overall pipeline.

3.1 Problem Definition – Compositional Multi-Agent Orchestration

Let $x \in \mathcal{X}$ denote an input task instance. We learn an orchestration policy π_θ over executable orchestration specifications:

$$y \sim \pi_\theta(\cdot | x), \quad y \in \mathcal{Y}, \quad (1)$$

where $\mathcal{Y}$ denotes the space of executable orchestration specifications. Each specification y induces an orchestration graph

$$G = \mathcal{C}(y) = (V, E), \quad (2)$$

where $V = \{v_1, \dots, v_n\}$ is the set of instantiated agents and $E \subseteq V \times V$ is the dependency structure. Each agent $v \in V$ is associated with a role specification R_v and a capacity level L_v . We write

$$R_v = (a_v, b_v, d_v), \quad (3)$$

where a_v denotes the task-specific agent identity, b_v denotes the inherited base role, and d_v denotes the customized duty. The capacity level $L_v \in \mathcal{L}$ specifies the worker model assigned to agent v . An edge $(u, v) \in E$ indicates that the output of agent u is provided as context to agent v during execution.

The learning problem is to optimize π_θ such that the generated specification is effective and efficient after execution. Formally, we maximize

$$\mathcal{J}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [\mathcal{U}(x, y)], \quad (4)$$

where $\mathcal{U}(x, y)$ denotes the execution utility of specification y on task x , incorporating task performance and execution cost. Section 3.4 specifies the orchestration reward and localized counterfactual objective used to optimize this utility.

3.2 Supervised Fine-Tuning of the Orchestration Policy

Before reinforcement learning, we fine-tune the orchestration policy on teacher-generated executable orchestration specifications. Without this initialization, the policy has difficulty following the required specification format, so early reinforcement learning is often dominated by invalid or non-executable rollouts rather than useful orchestration exploration. For each input x_i , a stronger teacher model generates a candidate serialized specification $\tilde{y}_i$ in the YAML format. We retain only candidates that satisfy the YAML/schema constraints and can be compiled into orchestration graphs, yielding a filtered SFT dataset $\mathcal{D}_{\text{sft}} = \{(x_i, y_i)\}$, where each y_i is a serialized executable orchestration specification.

Given a filtered pair $(x, y) \in \mathcal{D}_{\text{sft}}$, we treat the serialized specification $y = (y_1, \dots, y_{|y|})$ as the target sequence and optimize the teacher-forced likelihood:

$$\mathcal{J}_{\text{sft}}(\theta) = \mathbb{E}_{(x, y) \in \mathcal{D}_{\text{sft}}} \left[\sum_{t=1}^{|y|} \log \pi_\theta(y_t | x, y_{<t}) \right]. \quad (5)$$

The resulting policy initializes reinforcement learning and reduces invalid specifications during exploration.

3.3 Multi-Agent Orchestration Generation and Execution

Compositional orchestration generation. Given an input task x , the orchestration policy samples a serialized specification $y \sim \pi_\theta(\cdot | x)$. In our implementation, y is represented in a step-structured YAML format. Each step contains one or more agent entries, and each agent entry specifies the fields

corresponding to its role specification R_v , capacity level L_v , and dependency references to earlier agents. These references define the dependency structure E in the compiled orchestration graph, rather than requiring a separate edge list. A concrete YAML example of an executable orchestration specification is provided in Appendix A.

Before execution, the YAML specification is parsed and validated, with validity denoted by $\text{Valid}(y) \in \{0, 1\}$. A valid specification is compiled into an orchestration graph $G = \mathcal{C}(y)$, while an invalid specification is treated as an invalid rollout during training.

Orchestration execution. For a valid orchestration graph $G = (V, E)$, agents are executed according to the dependency order induced by the reference fields. Let $\text{pa}(v) = \{u : (u, v) \in E\}$ denote the parent agents of v . During execution, each agent v receives the input task x and the outputs of its parent agents, and produces an agent output o_v :

$$o_v = f_{L_v}(x, \{o_u : u \in \text{pa}(v)\}; R_v), \quad (6)$$

where f_{L_v} denotes the worker model associated with capacity level L_v , and R_v specifies the role specification of agent v . The final answer is obtained as $\hat{a} = h(G, x)$ from the terminal agent output or a terminal aggregation rule. Besides the final answer, execution records worker-token usage, agent count, and dependency count, which are used to compute the efficiency component of the orchestration-level reward.

3.4 Reinforcement Learning for Multi-Agent Orchestration

After supervised fine-tuning, we further optimize the orchestration policy with GRPO using rewards computed from executed specifications. For each task, sampled specifications are executed following Section 3.3, and the execution results and statistics define an orchestration reward for whole-specification optimization. However, because this reward is assigned to the complete serialized specification, it provides limited credit assignment for individual role, capacity, and dependency fields. We therefore introduce localized counterfactual credit assignment to localize reward differences to edited orchestration fields.

Global orchestration reward. For each task x , the orchestration policy samples a group of B serialized specifications:

$$y_i \sim \pi_\theta(\cdot \mid x), \quad i = 1, \dots, B. \quad (7)$$

Following the generation and execution procedure in Section 3.3, each valid sampled specification yields a final answer $\hat{a}_i$ and execution records η_i , including whether the answer is correct, the number of worker tokens consumed, the number of instantiated agents, and the number of dependencies. Invalid specifications are assigned an invalid-specification reward. We define the orchestration reward as:

$$R_{\text{orch}}(x, y_i) = \begin{cases} R_{\text{task}}(x, \hat{a}_i) + \lambda_{\text{tok}} B_{\text{tok}}(x, G_i) - \lambda_{\text{graph}} C_{\text{graph}}(G_i), & \text{Valid}(y_i) = 1, \\ r_{\text{invalid}}, & \text{Valid}(y_i) = 0, \end{cases} \quad (8)$$

where $G_i = \mathcal{C}(y_i)$ when y_i is valid. R_{task} measures task success, B_{tok} rewards worker-token efficiency, C_{graph} penalizes structural cost such as agent count and dependency count, and $r_{\text{invalid}} < 0$ penalizes invalid specifications.

Given the group rewards, GRPO computes the group-relative advantage:

$$\hat{A}_i = \frac{R_{\text{orch}}(x, y_i) - \bar{R}_x}{\sigma_x + \epsilon}, \quad \bar{R}_x = \frac{1}{B} \sum_{j=1}^B R_{\text{orch}}(x, y_j), \quad (9)$$

where σ_x is the standard deviation of $\{R_j\}_{j=1}^B$ and ϵ is a small constant for numerical stability. The serialized specification $y_i = (y_{i,1}, \dots, y_{i,|y_i|})$ is then optimized with the standard token-level GRPO objective using $\hat{A}_i$. This global objective reinforces complete executable orchestration specifications according to their execution utility.

Localized counterfactual credit assignment. Global orchestration-level GRPO reinforces complete serialized specifications, but it assigns the same specification-level advantage to all generated tokens. It therefore does not directly distinguish which role, capacity, or dependency field is responsible for a reward difference. We address this limitation by constructing counterfactual specifications: for a valid sampled specification y , we edit one orchestration field, execute the resulting counterfactual specification, and compare its reward with the original.

To align counterfactual edits with the three orchestration decisions, we define three mutation families $\mathcal{M} = \{m_E, m_R, m_L\}$: m_E removes one dependency reference to test whether an information link is necessary, m_R rolls back one customized duty to its inherited base-role description to test the value of role specialization, and m_L downgrades one capacity level to test whether the assigned capacity is needed. Each mutation identifies the edited token spans $(S, \tilde{S})$ in the original and counterfactual serialized specifications. We maintain a sampling probability p_m for each mutation family, initialized uniformly and updated from previous reward contrasts. For each valid specification with compiled graph $G = \mathcal{C}(y)$, we compute the feasible mutation set $\mathcal{M}(G) \subseteq \mathcal{M}$ and sample m_c by renormalizing $\{p_m\}_{m \in \mathcal{M}}$ over $\mathcal{M}(G)$. The original and counterfactual specifications are evaluated with the same orchestration reward, yielding:

$$\Delta_c = R_{\text{orch}}(x, y) - R_{\text{orch}}(x, \tilde{y}). \quad (10)$$

We set the preference direction as $b_c = 1$ if $\Delta_c \geq 0$ and $b_c = -1$ otherwise. Algorithm 1 summarizes the operational procedure, including feasible mutation selection, reward comparison, span extraction, and adaptive mutation sampling.

Here, u_m is the running contrast estimate for mutation family m , initialized to zero. The update rate α controls how quickly the estimate adapts to new comparisons, τ controls the softmax temperature, and $p_{\min}$ is the probability floor that preserves exploration.

Given the original and counterfactual edited spans, we score only the edited tokens while keeping the remaining serialized specification as conditioning context. For a serialized specification y and an edited span S , we define:

$$s_\theta(x, y, S) = \frac{1}{|S|} \sum_{t \in S} \log \pi_\theta(y_t \mid x, y_{<t}). \quad (11)$$

We write $s_\theta^{\text{orig}} = s_\theta(x, y, S)$ and $s_\theta^{\text{cf}} = s_\theta(x, \tilde{y}, \tilde{S})$. Tokens outside the edited span provide context but do not contribute to the counterfactual objective. Thus, unlike sequence-level preference learning over the entire serialized specification, this comparison localizes the update to the orchestration field whose edit produced the reward contrast.

The localized counterfactual objective is

$$\mathcal{J}_{\text{cf}}(\theta) = \mathbb{E} \left[w(\Delta_c) \log \sigma \left(\beta_{\text{cf}} b_c \left[s_\theta^{\text{orig}} - s_\theta^{\text{cf}} \right] \right) \right], \quad (12)$$

where β_{cf} controls the comparison sharpness and

$$w(\Delta_c) = \frac{\min(|\Delta_c|, \delta_{\max})}{\delta_{\max}} \quad (13)$$

downweights small or saturated reward contrasts.

Overall optimization. The localized counterfactual objective is applied within the same policy optimization stage as GRPO. Specifically, GRPO uses R_{orch} to assign complete-specification advantages, while $\mathcal{J}_{\text{cf}}$ reuses the same reward to form counterfactual reward contrasts and applies the resulting preference only to the edited span. We weight this span-level update by λ_{cf} in implementation. Thus, the local term changes the granularity of credit assignment rather than introducing a separate local reward.

Algorithm 1 Localized counterfactual credit assignment

Require: Task x , valid specification y , mutation probabilities $\{p_m\}_{m \in \mathcal{M}}$, running contrast estimates $\{u_m\}_{m \in \mathcal{M}}$

- 1: Compile $G = \mathcal{C}(y)$ and evaluate $R_{\text{orch}}(x, y)$
- 2: Compute feasible mutation set $\mathcal{M}(G) \subseteq \mathcal{M}$
- 3: Sample m_c by renormalizing $\{p_m\}$ over $\mathcal{M}(G)$
- 4: Sample a valid edit location ℓ for mutation type m_c
- 5: Apply the edit to obtain $\tilde{y}$ and edited spans $(S, \tilde{S})$
- 6: Evaluate $R_{\text{orch}}(x, \tilde{y})$
- 7: $\Delta_c \leftarrow R_{\text{orch}}(x, y) - R_{\text{orch}}(x, \tilde{y})$
- 8: $b_c \leftarrow 1$ if $\Delta_c \geq 0$, otherwise -1
- 9: $u_{m_c} \leftarrow (1 - \alpha)u_{m_c} + \alpha|\Delta_c|$
- 10: $\mathbf{p} \leftarrow \text{FloorSoftmax}_{p_{\min}}(\mathbf{u}/\tau)$
- 11: **return** $(y, \tilde{y}, S, \tilde{S}, \Delta_c, b_c)$

Table 1: Performance comparison (%) across six benchmarks. Avg. is computed across the six reported benchmarks.

Method	MMLU [†]	GSM8K	AQuA	MultiArith	SVAMP	HumanEval [‡]	Avg.
<i>Single-Agent Reasoning</i>							
Vanilla	74.46	85.59	46.46	76.00	80.80	80.64	73.99
CoT	66.46	91.36	81.89	98.67	93.60	81.45	85.57
SC-CoT	67.54	93.25	82.68	98.83	94.40	<u>83.88</u>	86.76
<i>Fixed Multi-Agent Orchestration</i>							
Chain-MAS	78.51	92.19	77.17	98.83	94.40	80.65	86.96
Tree-MAS	79.56	92.49	82.68	98.67	93.90	<u>83.88</u>	<u>88.53</u>
Complete-MAS	79.16	92.65	83.46	98.83	94.00	82.25	88.39
Random-MAS	79.69	92.19	81.10	98.67	94.00	83.06	88.12
<i>Adaptive Workflow Orchestration</i>							
AFlow	70.35	91.05	<u>83.86</u>	96.33	93.80	76.83	85.37
<i>Topology Design</i>							
AgentDropout	80.73	92.84	83.07	97.50	93.10	77.42	87.44
AgentPrune	80.54	91.36	<u>83.86</u>	98.00	<u>95.20</u>	79.84	88.13
G-Designer	78.25	92.17	82.28	98.17	93.10	71.77	85.96
ARG-Designer	85.30	<u>93.35</u>	69.16	<u>98.93</u>	94.90	76.61	86.38
OFA-MAS	78.38	84.84	82.67	<u>98.67</u>	94.20	79.84	86.43
<i>Compositional Orchestration Generation</i>							
Ours	<u>84.32</u>	94.32	86.22	99.17	95.60	84.68	90.72

[†]MMLU is evaluated on the validation set with 1,531 examples rather than the full test set. [‡]HumanEval is evaluated on the held-out 124-example split for all methods; the remaining 40 examples are used only for training/adaptation by methods that require it. Full evaluation protocol is listed in Appendix D.7.

4 Experiments

We evaluate LEMON across reasoning and coding tasks. We compare task performance against single-agent prompting, fixed-structure MAS, topology-design, and adaptive workflow baselines, and further analyze the accuracy–token trade-off of the learned orchestration. We also ablate supervised initialization, global reinforcement learning, role–capacity–dependency generation, and localized counterfactual mutations, followed by case studies and training-dynamic analysis of the generated specifications.

4.1 Experimental Setup

Benchmarks. We evaluate on six benchmarks covering mathematical reasoning, multiple-choice reasoning, and code generation. The mathematical reasoning benchmarks include GSM8K [26], MultiArith [27], SVAMP [28], and AQuA [29]. The multiple-choice reasoning benchmark is MMLU [30], and the code-generation benchmark is HumanEval [31]. For each task instance, the orchestrator generates a structured multi-agent orchestration, which is then executed to produce the final answer.

Baselines. We compare our method with several representative baselines. The single-agent baselines include Vanilla, CoT [32], and SC(CoT) [33]. The fixed-structure MAS baselines include Chain, Tree, Complete, and Random. The topology-design baselines include AgentPrune [9], AgentDropout [34], G-Designer [8], ARG-Designer [15], and OFA-MAS [16]. The adaptive MAS baselines include AFlow [10].

Performance. Table 1 reports the main performance comparison across six benchmarks. Our method achieves the best average score of 90.72, outperforming the strongest single-agent baseline SC-CoT by 3.96 points, the strongest fixed-structure MAS baseline Tree-MAS by 2.19 points, the adaptive workflow baseline AFlow by 5.35 points, and the strongest topology-design baseline AgentPrune by 2.59 points. It obtains the best performance on five out of six benchmarks, including GSM8K, AQuA, MultiArith, SVAMP, and HumanEval, and achieves the second-best result on MMLU, only 0.98 points below ARG-Designer. These results indicate that generating task-specific multi-agent orchestration is more effective than single-agent prompting, fixed communication structures, or partial

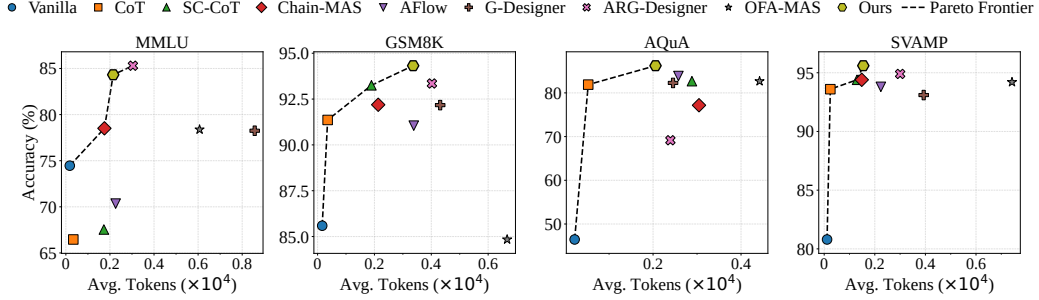


Figure 3: The worker token cost comparison.

orchestration adaptation. They also support our central claim that role specifications, capacity levels, and dependency structure should be generated jointly as a single executable specification.

Token Efficiency. Beyond task performance, we evaluate the accuracy–token trade-off of different methods. Figure 3 plots task accuracy against average worker-token usage on four representative benchmarks, with dashed curves denoting the Pareto frontier. LEMON lies on or close to the Pareto frontier across the plotted tasks. On MMLU, it achieves competitive accuracy while using about 0.2×10^4 worker tokens, compared with roughly 0.3×10^4 for ARG-Designer and more than 0.6×10^4 for several topology-design baselines. On AQuA and SVAMP, LEMON reaches the highest-accuracy region with substantially fewer tokens than high-cost topology baselines such as OFA-MAS. On GSM8K, it also achieves the best accuracy without moving to the high-token end of the plot. These results suggest that generating role specifications, capacity levels, and dependency structure as one executable specification helps the orchestrator allocate worker computation more selectively.

Ablation. Table 2 reports ablations on MMLU, GSM8K, and AQuA. Compared with Global RL Only, LEMON improves accuracy from 83.08/93.40/85.43 to 84.32/94.32/86.22, while reducing worker tokens from 2543/3743/3013 to 2148/3362/2056. Removing role generation, capacity allocation, or structure generation consistently hurts accuracy, with the largest drops appearing for structure generation on MMLU (-3.46) and role generation on AQuA (-3.15). Capacity allocation is especially important for efficiency: removing it increases token usage to 3401/4311/4154. Removing each counterfactual mutation family also degrades performance, showing that dependency-, role-, and capacity-level counterfactual edits all contribute useful local credit. Finally, w/o SFT suffers the largest degradation, dropping to 48.53/75.82/64.96 accuracy, confirming that supervised warm start is important for stable RL exploration.

Table 2: Ablation studies on representative benchmarks.

Method	MMLU		GSM8K		AQuA	
	Acc.	Tok.↓	Acc.	Tok.↓	Acc.	Tok.↓
Global RL Only	83.08	2543.21	93.40	3742.59	85.43	3012.54
Ours	84.32	2148.00	94.32	3362.00	86.22	2056.10
w/o Role Gen.	82.43	2496.77	92.72	3734.76	83.07	2829.91
w/o Cap. Alloc.	82.76	3401.06	93.18	4310.98	84.65	4154.38
w/o Struct. Gen.	80.86	2347.71	93.33	3991.04	83.86	3296.94
w/o Struct. CF	83.54	2516.13	93.78	3797.33	85.43	3303.37
w/o Role CF	83.67	2484.87	93.93	3553.81	85.83	2680.48
w/o Cap. CF	84.00	2663.06	94.09	4362.87	85.83	3589.65
w/o SFT	48.53	2777.60	75.82	3964.21	64.96	3729.20

Case Study. Figure 4 provides qualitative examples on GSM8K and MMLU. Compared with ARG-DESIGNER and OFA-MAS, LEMON generates more task-aligned orchestration structures rather than only adapting the graph topology. In the GSM8K example, the generated agents correspond to the main reasoning steps: analyzing the orange distribution, computing the remaining oranges, and verifying the final answer. In the MMLU example, LEMON decomposes the medical question into parsing the neural tube defect diagnosis target, comparing prenatal tests, identifying the exception among valid diagnostic methods, and producing the final answer. The expanded view further shows that the generated customized duties and dependency references follow these reasoning steps, while capacity levels are assigned according to the expected burden of each agent. In contrast, ARG-DESIGNER and OFA-MAS often use broader or repeated roles, such as *Inspector*, *Critic*, and *Knowledgeable Expert*, whose responsibilities are less directly tied to the local task requirements. These cases illustrate that LEMON jointly aligns role specifications, capacity levels, and dependency structure in the executable orchestration.

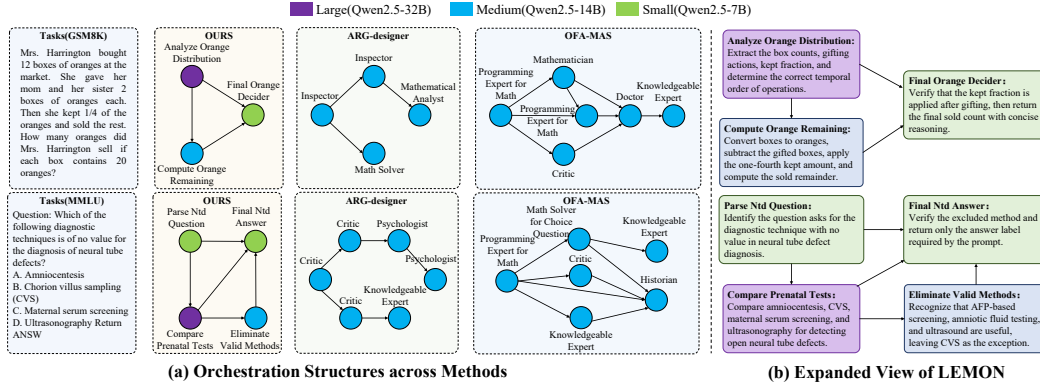


Figure 4: Case studies on GSM8K and MMLU. Node colors denote capacity levels, and the expanded view shows LEMON’s customized duties and dependency references.

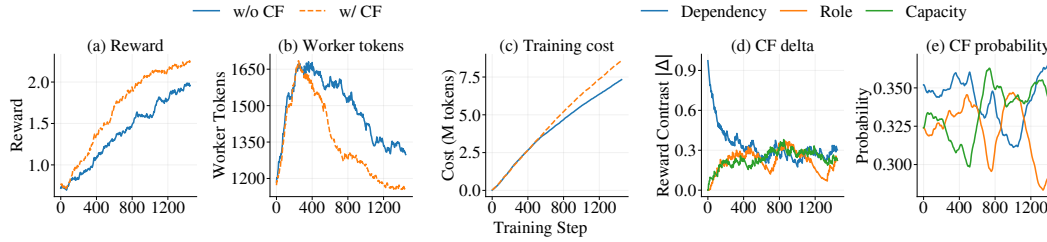


Figure 5: Effect of local counterfactual credit assignment on reward, efficiency, and mutation dynamics.

Effect of local counterfactual credit assignment. We analyze how local counterfactual credit assignment affects training dynamics in Figure 5. Compared with a GRPO-only variant trained with the same hyperparameter settings except for removing the localized counterfactual objective, the full method achieves consistently higher reward, increasing the final reward from roughly 2.0 to 2.25. This suggests that local counterfactual comparisons provide useful optimization signal beyond whole-specification reinforcement.

The method also exhibits a clear training–execution trade-off. Since each valid rollout is paired with one edited counterfactual specification, cumulative training cost increases from about 7.4M to 8.3M tokens. This cost is not doubled because unchanged agent-node executions are reused through a node-level execution cache; implementation details are provided in Appendix C. In return, the learned policy generates more token-efficient specifications, reducing final worker-token usage from about 1.30K to 1.15K tokens per execution. The counterfactual statistics further show that dependency deletion gives the strongest early reward contrast, while role rollback and capacity downgrade remain active later in training. Meanwhile, mutation probabilities adapt over time rather than staying fixed, indicating that the sampler reallocates local supervision across dependency, role, and capacity decisions as their reward contrasts change.

5 Conclusion

We presented LEMON, a framework for learning executable multi-agent orchestration. LEMON generates an executable orchestration specification that jointly determines task-specific role specifications, capacity levels, and dependency structure, instead of optimizing these decisions separately. The orchestration policy is trained with orchestration-level GRPO and localized counterfactual credit assignment, which applies reward contrasts only to edited role, capacity, or dependency spans. Experiments across reasoning and coding benchmarks show that LEMON improves over single-agent prompting, fixed multi-agent structures, topology-design methods, and adaptive workflow baselines. Ablations and training-dynamic analyses further validate the benefits of compositional orchestration generation and localized counterfactual credit assignment.

References

- [1] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- [2] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in neural information processing systems*, 36:51991–52008, 2023.
- [3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.
- [4] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- [5] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first international conference on machine learning*, 2024.
- [6] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- [7] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.
- [8] Guibin Zhang, Yanwei Yue, Xiangguo Sun, Guancheng Wan, Miao Yu, Junfeng Fang, Kun Wang, Tianlong Chen, and Dawei Cheng. G-designer: Architecting multi-agent communication topologies via graph neural networks. *arXiv preprint arXiv:2410.11782*, 2024.
- [9] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024.
- [10] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- [11] Stefan Nielsen, Edoardo Cetin, Peter Schwendeman, Qi Sun, Jinglue Xu, and Yujin Tang. Learning to orchestrate agents in natural language with the conductor. *arXiv preprint arXiv:2512.04388*, 2025.
- [12] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Workflowllm: Enhancing workflow orchestration capability of large language models. *arXiv preprint arXiv:2411.05451*, 2024.
- [13] Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025.
- [14] Shuowei Cai, Yansong Ning, and Hao Liu. Agentbalance: Backbone-then-topology design for cost-effective multi-agent systems under budget constraints. *arXiv preprint arXiv:2512.11426*, 2025.
- [15] Shiyuan Li, Yixin Liu, Qingsong Wen, Chengqi Zhang, and Shirui Pan. Assemble your crew: Automatic multi-agent communication topology design via autoregressive graph generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 23142–23150, 2026.

- [16] Shiyuan Li, Yixin Liu, Yu Zheng, Mei Li, Quoc Viet Hung Nguyen, and Shirui Pan. Ofa-mas: One-for-all multi-agent system topology design based on mixture-of-experts graph generative models. In *Proceedings of the ACM Web Conference 2026*, pages 1333–1344, 2026.
- [17] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [18] Jinwei Su, Qizhen Lan, Yinghui Xia, Lifan Sun, Weiyu Tian, Tianyu Shi, and Lewei He. Difficulty-aware agentic orchestration for query-specific multi-agent workflows. In *Proceedings of the ACM Web Conference 2026*, pages 2060–2070, 2026.
- [19] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. Masrouter: Learning to route llms for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572, 2025.
- [20] Tianjun Yao, Zhaoyi Li, and Zhiqiang Shen. Hieramas: Optimizing intra-node llm mixtures and inter-node topology for multi-agent systems. *arXiv preprint arXiv:2602.20229*, 2026.
- [21] Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, et al. Toolorchestra: Elevating intelligence via efficient model and tool orchestration. *arXiv preprint arXiv:2511.21689*, 2025.
- [22] Hongcheng Gao, Yue Liu, Yufei He, Longxu Dou, Chao Du, Zhijie Deng, Bryan Hooi, Min Lin, and Tianyu Pang. Flowreasoner: Reinforcing query-level meta-agents. *arXiv preprint arXiv:2504.15257*, 2025.
- [23] Siyu Wang, Ruotian Lu, Zhihao Yang, Yuchao Wang, Lei Xu, Qimin Xu, Guojun Yin, Cailian Chen, Xinping Guan, et al. Topoweaver-r1: Reinforcing difficulty-aware topology evolution in multi-agent competition-level code generation.
- [24] Haoning Jiang, Han Wu, Zhuoli Ouyang, Ziheng Wang, Tinghuan Chen, and Junmin Jiang. Fd-magpro: Functionality-driven multi-agent group relative policy optimization for analog-ldo sizing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 22310–22317, 2026.
- [25] Baoheng Zhu, Deyu Bo, Delvin Ce Zhang, and Xiao Wang. Graph-grpo: Training graph flow models with reinforcement learning. *arXiv preprint arXiv:2603.10395*, 2026.
- [26] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [27] Subhro Roy and Dan Roth. Solving general arithmetic word problems. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 1743–1752, 2015.
- [28] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are nlp models really able to solve simple math word problems? In *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 2080–2094, 2021.
- [29] Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In *Proceedings of the 55th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 158–167, 2017.
- [30] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [31] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

- [32] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [33] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [34] Zhexiong Wang, Yutong Wang, Xuebo Liu, Liang Ding, Miao Zhang, Jie Liu, and Min Zhang. Agentdropout: Dynamic agent elimination for token-efficient and high-performance llm-based multi-agent collaboration. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24013–24035, 2025.

A Example YAML Orchestration Specification

The following example shows the YAML format used to serialize an executable orchestration specification. Each agent entry specifies a task-specific role identifier, an inherited base role, a customized duty, dependency references to earlier agents, and a capacity level. The duty descriptions in this example are generic templates provided only to illustrate the field format; in our experiments, duties are generated by the orchestration policy for each input task.

```
defaults:
  capacity: medium
steps:
  - agents:
    - type: extract_quantities
      base_role: quantity_extractor
      duty: Extract known quantities, unknown target, and explicit constraints.
      ref: []
      capacity: small

    - agents:
      - type: build_equations
        base_role: equation_builder
        duty: Translate extracted quantities into arithmetic relations.
        ref: [extract_quantities]
        capacity: medium
      - type: check_units
        base_role: unit_checker
        duty: Verify that each number is attached to the correct entity or unit.
        ref: [extract_quantities]
        capacity: small

    - agents:
      - type: compute_answer
        base_role: calculator
        duty: Perform the arithmetic from the equation and unit check.
        ref: [build_equations, check_units]
        capacity: medium

    - agents:
      - type: verify_final_answer
        base_role: verifier
        duty: Return the final answer after checking referenced arithmetic.
        ref: [compute_answer, check_units]
        capacity: medium
```

B Limitations

LEMON has two main limitations. First, it requires reinforcement learning over executed orchestration specifications, which introduces non-trivial training-time cost. In particular, the localized counterfactual objective evaluates one counterfactual variant for each valid sampled specification. This overhead is limited to training, and inference only requires generating and executing one orchestration specification. Second, our experiments focus on text-based reasoning and coding benchmarks, including MMLU, GSM8K, AQUA, MultiArith, SVAMP, and HumanEval. Although these benchmarks cover multiple task formats, they do not exhaustively represent broader multi-agent scenarios such as long-horizon tool use, multimodal inputs, or interactive environments. We leave broader evaluation under more diverse task settings to future work.

C Counterfactual Overhead and Execution Caching

Localized counterfactual credit assignment introduces additional training-time computation because each valid sampled orchestration specification is paired with one counterfactual variant. A naive implementation would execute both the original specification and the edited counterfactual specifica-

tion independently. This would substantially increase worker-execution cost for valid rollouts, even though each counterfactual edit modifies only one local orchestration field.

To reduce redundant execution, our implementation uses node-level execution caching during training. The cache stores worker outputs for executed agent calls together with their deterministic execution context, including the input task, the selected base role, the customized duty, the capacity level, and the referenced upstream outputs. When constructing a counterfactual specification, only the edited field and its downstream dependent agents may require recomputation. Agents whose prompts and upstream contexts remain unchanged are reused from the cache rather than executed again. This caching mechanism does not change the reward definition or the counterfactual objective; it only avoids redundant worker-model calls.

Let $G = C(y)$ be the original orchestration graph and $\tilde{G} = C(\tilde{y})$ be the counterfactual graph obtained by applying one local mutation. Without caching, evaluating the counterfactual would require executing all valid agents in $\tilde{G}$. With node-level execution caching, only agents whose prompts or upstream contexts change under the counterfactual edit need to be recomputed. Therefore, unchanged agents can be reused from the original execution whenever their role, duty, capacity level, and referenced upstream outputs remain the same.

For dependency deletion, the affected agents are usually downstream agents that directly or indirectly consume the removed reference. For role rollback or capacity downgrade, the edited agent and its downstream dependents may require recomputation, while unrelated branches remain reusable.

We further bound the overhead by sampling only one feasible mutation type and one edit location for each valid rollout, rather than enumerating all possible dependency, role, and capacity edits. Counterfactual evaluation is used only during reinforcement learning. At inference time, LEMON generates one orchestration specification and executes it once, so localized counterfactual evaluation introduces no inference-time cost.

D Reproducibility Details

D.1 Executable Orchestration Specification

LEMON represents each orchestration as a structured YAML specification. The orchestration policy does not directly answer the task. Instead, it generates an executable multi-agent specification that defines the agents to instantiate, their role specifications, their capacity levels, and their dependency references. Each agent entry contains a task-specific role identifier type, an inherited `base_role`, a customized natural-language duty, a list of dependency references `ref`, and a capacity field.

The validator requires each active agent to include `type`, `base_role`, `duty`, `ref`, and `capacity`. The capacity level must be one of `small`, `medium`, or `large`. References in the first step must be empty, and references in later steps can only point to active agents from earlier steps. Valid specifications are compiled into layered directed acyclic graphs, and agents in the same step are executed concurrently. This validation step ensures that each sampled specification can be deterministically mapped to an executable orchestration graph before worker execution.

D.2 Model Deployment

Table 3 reports the model backbones used by the orchestration policy and worker agents. The orchestration policy is initialized from a 7B instruction model and is trained to generate executable specifications. During execution, the generated capacity level selects one of three worker backbones. The small, medium, and large capacity levels correspond to increasingly stronger Qwen2.5-Instruct models.

This deployment allows the orchestrator to allocate different worker capacities within the same generated specification, rather than using a homogeneous worker model for all agents.

D.3 Main Training Hyperparameters

Table 4 summarizes the main optimization and generation hyperparameters used for reinforcement learning. We train for 1500 steps with at most 480 RL training samples per dataset. For each task, the

Table 3: Model deployment used in our experiments.

Role	Model
Orchestration policy backbone	Qwen/Qwen2.5-7B-Instruct
Small-capacity worker	Qwen/Qwen2.5-7B-Instruct
Medium-capacity worker	Qwen/Qwen2.5-14B-Instruct
Large-capacity worker	Qwen/Qwen2.5-32B-Instruct

policy samples a group of candidate orchestration specifications, executes valid candidates, and uses group-relative rewards for policy optimization. The prompt and generation length limits are set to 2048 tokens to allow the policy to output complete YAML specifications while keeping rollout cost bounded.

Table 4: Main training hyperparameters.

Hyperparameter	Value
Seed	42
Training steps	1500
Max train samples	480
Split preset	cross_dataset_v1
Group size / generations	4
Prompts per reward call	4
Candidate rows per reward call	16
Learning rate	1×10^{-6}
LoRA	enabled
LoRA rank / alpha	8 / 32
Gradient checkpointing	enabled
Train batch size	4
Gradient accumulation	1
Max prompt tokens	2048
Max new tokens	2048
Temperature / top- p	0.6 / 0.9
Role pool variant	baseline
Node cache	enabled

The group size controls the number of sampled specifications used to compute group-relative advantages for each task. The values for prompts per reward call and candidate rows per reward call determine the rollout batching structure: each reward call evaluates four prompts and four generations per prompt, yielding sixteen candidate rows. LoRA and gradient checkpointing are used to reduce memory usage during policy optimization. Node caching avoids repeated worker execution for identical cached agent calls when applicable.

D.4 Reward Hyperparameters

Table 5 reports the weights used in the orchestration-level reward. The reward combines execution correctness, efficiency, and structural cost. The execution term measures whether the generated orchestration solves the task, the efficiency term rewards lower worker-token usage, and the structure term penalizes unnecessarily complex orchestration graphs.

Table 5: Reward hyperparameters.

Hyperparameter	Value
efficiency_weight	1.5
structure_weight	0.1
execution_weight	1.0

The execution weight is the main task-performance coefficient. The efficiency weight encourages the orchestrator to avoid excessive worker-token usage when a more compact execution is sufficient. The

structure weight is smaller because graph complexity is treated as a regularization term rather than the primary optimization target.

D.5 Localized Counterfactual Hyperparameters

The localized counterfactual objective is enabled during reinforcement learning. For each valid sampled specification, the training loop samples at most one feasible counterfactual pair per reward call. The three mutation families are dependency deletion, role rollback, and capacity downgrade. Table 6 lists the hyperparameters controlling mutation sampling, reward-contrast shaping, and the strength of the counterfactual objective.

Table 6: Localized counterfactual training hyperparameters.

Hyperparameter	Value
Localized counterfactual objective	enabled
Mutation families	dependency deletion, role rollback, capacity downgrade
Dependency deletion count	1
Role rollback count	1
Capacity downgrade count	1
Sampling mode	adaptive_rotation
Delta mode	typewise_group_shaped
EMA alpha	0.1
Rotation temperature	1.0
Minimum type probability	0.05
Counterfactual objective weight	0.05
Preference beta	0.1
Delta cap	0.5
Minimum absolute delta	0.01

The mutation counts are set to one so that each counterfactual pair changes only one local orchestration field. This keeps the reward contrast attributable to a specific dependency, role, or capacity decision. The adaptive rotation sampler adjusts the probability of choosing each mutation family according to its recent reward contrasts, while the minimum type probability preserves exploration over all mutation types. The delta cap and minimum absolute delta stabilize the counterfactual signal by clipping very large contrasts and filtering negligible reward differences. The counterfactual objective weight controls the contribution of localized counterfactual learning relative to the orchestration-level GRPO objective.

D.6 Supervised Warm Start

Before reinforcement learning, the orchestration policy is initialized from supervised fine-tuning on teacher-generated executable YAML specifications. This warm start reduces the frequency of invalid YAML outputs during early RL exploration and provides the policy with an initial prior over role, capacity, and dependency fields. We use 500 SFT samples and train a LoRA checkpoint with LLaMA-Factory. The resulting checkpoint is used to initialize the main GRPO training runs. The SFT run uses 32 global steps and reaches a training loss of 0.166656.

D.7 Evaluation Protocol

For RL training, we sample at most 480 examples from the training split of each dataset when an official training split is available. For HumanEval, which does not provide a separate training split, we use 40 examples for RL training or task-level adaptation by methods that require it; all methods evaluated on the remaining 124. For MMLU, we evaluate on the validation split because the official test split is substantially larger and more costly.

Evaluation is conducted on MMLU, GSM8K, AQuA, MultiArith, SVAMP, and HumanEval. For each evaluation instance, the orchestrator generates one executable orchestration specification. The validator checks schema validity and dependency validity, and the compiled orchestration graph is executed once to produce the final answer. Table 7 summarizes the evaluation split and task type for each benchmark.

Table 7: Evaluation datasets.

Dataset	Evaluation size	Task type
MMLU (val)	1531	multiple-choice reasoning
GSM8K (test)	1319	mathematical reasoning
AQuA (test)	254	mathematical reasoning
MultiArith	600	mathematical reasoning
SVAMP	1000	mathematical reasoning
HumanEval (held-out)	124	code generation

For mathematical reasoning and multiple-choice benchmarks, accuracy is computed by matching the extracted final answer against the ground-truth answer. For HumanEval, we report pass@1 under unit-test execution. Since the HumanEval evaluation uses the held-out 124 examples, the reported code-generation results do not overlap with the 40 HumanEval examples used for RL training.

D.8 Implementation Notes

The implementation uses role behavior templates rather than a universal agent tool-calling protocol. The generated specification selects the `base_role`, customized duty, capacity level, and dependency references. During execution, the runtime constructs each worker prompt from these fields and the outputs of referenced upstream agents. Tools can be registered in the runtime, but role behavior is not delegated to a general tool-call interface. This design keeps the execution protocol deterministic once the YAML specification has been generated and validated.

E Compute Resources

We report the compute resources used for training and evaluation in Table 8. Counterfactual evaluation introduces additional training-time execution cost because each valid sampled specification is paired with one counterfactual variant. This overhead is only used during training. At inference time, LEMON generates and executes one orchestration specification for each input task.

Table 8: Compute resources used in our experiments.

Item	Value
Compute platform	Amazon EC2
Instance type	<code>g6e.12xlarge</code>
GPU type	NVIDIA L40S
Number of GPUs	4
GPU memory	4 × 48 GB
CPU type	AMD EPYC 7R13
vCPUs	48
System memory	384 GiB
Storage used	1000 GB
Main RL training time	24 h 50 min

F Broader Impacts

LEMON aims to improve the effectiveness and efficiency of LLM-based multi-agent orchestration. Its potential positive impacts include reducing redundant worker-model calls, improving the efficiency of multi-agent reasoning systems, and enabling more systematic construction of task-adaptive orchestration specifications. These properties may make multi-agent systems easier to study, compare, and deploy under controlled resource budgets. As with other LLM-based systems, practical deployment should still include task-specific validation and appropriate human oversight.

G Existing Assets and Licenses

We use existing datasets, baseline implementations, and LLM backbones for research evaluation. We cite the original papers or repositories for these assets and follow their licenses or terms of use. For assets whose license is not explicitly specified in the local snapshot, we report the license status as not found and use the asset only for research comparison without redistributing it. Table 9 summarizes the main existing assets used in this work.

Table 9: Existing assets used in this work.

Asset	Type	Source / Version	License / Terms
MMLU	Dataset	Hendrycks et al.; local snapshot	MIT License
GSM8K	Dataset	Cobbe et al.; local snapshot	MIT License
AQuA	Dataset	Ling et al.; local snapshot	Apache License 2.0
MultiArith	Dataset	Roy and Roth; local snapshot	Not explicitly specified in local snapshot; research evaluation only
SVAMP	Dataset	Patel et al.; local snapshot	MIT License
HumanEval	Dataset	Chen et al.; local snapshot	MIT License
AFlow	Baseline / Code	Original repository; local snapshot	MIT License
G-Designer	Baseline / Code	Original repository; local snapshot	License not found in local snapshot; research evaluation only
OFA-MAS	Baseline / Code	Original repository; local snapshot	License not found in local snapshot; research evaluation only
ARG-DESIGNER	Baseline / Code	Original repository; local snapshot	License not found in local snapshot; research evaluation only
Qwen2.5-7B-Instruct	Model	Qwen/Qwen2.5-7B-Instruct	Apache License 2.0
Qwen2.5-14B-Instruct	Model	Qwen/Qwen2.5-14B-Instruct	Apache License 2.0
Qwen2.5-32B-Instruct	Model	Qwen/Qwen2.5-32B-Instruct	Apache License 2.0

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state the paper’s main claims: LEMON generates a final executable orchestration specification that compositionally determines task-specific roles, capacity levels, and dependency structure, and is trained with orchestration-level GRPO plus localized counterfactual credit assignment. These claims are supported by the method formulation in Section 3 and by experiments and ablations on six reasoning and coding benchmarks in Section 4.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We include a dedicated limitations discussion in Appendix B. The discussion describes the scope of the proposed method, its experimental coverage, and practical considerations for applying the learned orchestration policy.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include formal theoretical results such as theorems, lemmas, or propositions. The mathematical expressions in Section 3 are used to define the orchestration formulation and training objectives rather than to state theoretical claims.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We provide the information needed to reproduce the main experimental results, including the orchestration specification format, validation and execution procedure, training objectives, reward definition, datasets, baselines, evaluation metrics, and implementation details. Additional hyperparameters and reproduction details are provided in Appendix D.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide anonymized code and reproduction instructions for the main experiments in the supplemental material. The used benchmarks are publicly available, and we describe the data access, preprocessing, environment setup, and scripts needed to reproduce the reported results in Appendix D.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: We specify the experimental settings needed to understand the reported results, including the evaluated benchmarks, baselines, evaluation metrics, worker model configuration, orchestration execution protocol, and training setup. Full hyperparameters, decoding settings, and implementation details are provided in Appendix D.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We do not report error bars for all main experiments because training and evaluating multi-agent orchestration policies with multiple LLM workers is computationally expensive. We use the same evaluation split, worker model configuration, decoding setup, and scoring scripts across all compared methods, and report ablation results to support the main empirical conclusions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We report the compute resources used for training and evaluation in Appendix E, including the cloud platform, instance type, GPU type, number of GPUs, GPU memory, CPU type, vCPU count, system memory, storage used, and the wall-clock time for the main RL training run.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research conforms to the NeurIPS Code of Ethics. We use publicly available benchmarks, do not conduct human-subject experiments or collect sensitive personal data, and preserve anonymity in the submitted manuscript and released artifacts.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: We discuss broader impacts in Appendix F. The discussion states that LEMON may improve the effectiveness and efficiency of LLM-based multi-agent orchestration by reducing redundant worker-model calls and supporting more systematic construction of task-adaptive orchestration specifications. It also notes that practical deployment should include task-specific validation and appropriate human oversight, consistent with the use of LLM-based systems in real applications.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release high-risk data or models such as pre-trained language models, image generators, or scraped datasets. The released artifacts consist of anonymized research code, configuration files, and instructions for reproducing the experiments.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: We properly cite the original creators of the datasets, baseline methods, code packages, and LLM backbones used in the paper. We describe the versions, access sources, licenses or terms of use, and usage restrictions of these existing assets in Appendix G.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: We release anonymized research code, configuration files, and reproduction instructions as new assets accompanying the submission. The documentation describes the implementation, training and evaluation procedures, license, limitations, and usage instructions in the released package.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing experiments or research with human subjects. Therefore, there are no participant instructions, screenshots, or compensation details to report.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve research with human subjects. Therefore, IRB approval or equivalent ethics review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are core components of LEMON: the orchestration policy generates executable orchestration specifications, and the generated agents are executed by LLM workers. We describe their roles, worker configuration, capacity-level mapping, and decoding settings in Section 3.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.